WERX Staking

SMART CONTRACT AUDIT REPORT

IMMUNEBYTES

19 September 2023

Introduction **3**

About Uwerx 3

About ImmuneBytes 3

Project Overview **4**

Documentation Details **5**

Audit Goals **5**

Audit Process & Methodology **6**

Audit Details **7**

Security Level References **8**

Severity Status References **9**

Severity Status **9**

Findings **10**

Critical Severity Issues **11**

High Severity Issues **11**

Medium severity issues **11**

Low severity issues **15**

Informational **17**

Gas Optimization **17**

Concluding Remarks **18**

Disclaimer **18**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Uwerx

Uwerx aims to revolutionize and reconceptualize the freelance industry by launching the first global, blockchain-powered freelance platform with a host of unique features that will not only disrupt but also transform the traditional freelancing industry.

Visit https://www.uwerx.network/ to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting smart contract audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 255+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Project Overview

Uwerx is an innovative and secure blockchain-based freelancing platform that connects talented professionals with clients worldwide. Uwerx aims to revolutionize and reconceptualize the freelance industry by launching the first global, blockchain-powered freelance platform with a host of unique features that will disrupt and transform the traditional freelancing industry.

One of the key features of Uwerx is its use of blockchain technology, which ensures secure and transparent transactions between freelancers and clients. All payments are made through Uwerx's cryptocurrency, which provides an additional layer of security and transparency.

Uwerx is committed to providing a safe and trustworthy platform for freelancers and clients. Our platform includes features such as escrow payments, dispute resolution, performance and sign-up incentives, freelancer webinars, and a rating system to ensure that freelancers and clients have a positive experience.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. https://uwerx.gitbook.io/uwerx-information/

Audit Goals

The focus of the audit was to verify that the smart contract system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within each contract and within the system of contracts.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established smart contract best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include

    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

The ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is a structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors, which includes -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

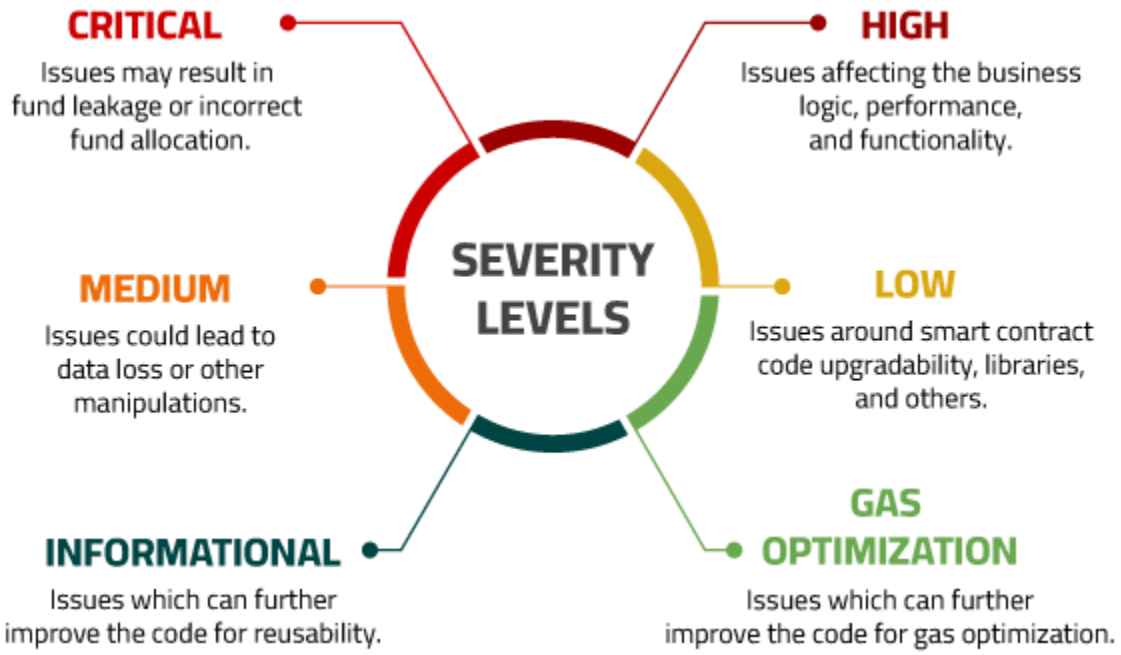
Audit Details

Project Name	Uwerx
Platform	Polygon
Contract	Deployed Contract Link - Fixed Contract Link
Name of Contract	WERX Staking
Languages	Solidity
Auditors	Gul Hameed | Kaif Ahmed
Report Type	Final

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	-	-
Fixed	-	-	3	-	-	-
Redacted	-	-	-	-	-	-
Acknowledged	-	-	4	4	-	-

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Findings

ID	Findings	Severity	Status
WSC - 001	No Withdraw Function for Owner	Medium	Fixed
WSC - 002	No Emergency Unstake Function	Medium	Acknowledged
WSC - 003	No Re-Entrancy Guard	Medium	Fixed
WSC - 004	No Function to Update stakedToken	Medium	Acknowledged
WSC - 005	No Checks for Zero Address	Medium	Fixed
WSC - 006	No Time Lock for Owner-Only Functions	Medium	Acknowledged
WSC - 007	No Penalty for Early Unstaking	Medium	Acknowledged
WSC - 008	No Function to Query Unclaimed Stakes	Low	Acknowledged
WSC - 009	No Function to Update Staking Periods	Low	Acknowledged
WSC - 010	No Function to Retrieve Staking Plans	Low	Acknowledged
WSC - 011	No Cap on Staking	Low	Acknowledged

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

No issues were found.

Medium severity issues

WSC - 001	No Withdraw Function for Owner		
Function Name	-		
Rating	Severity: Medium	Likelihood:	Impact:

Description
The contract currently lacks a function that allows the contract owner to withdraw any tokens or ETH that may have been sent to the contract by mistake.

Impact
Without this function, any funds sent to the contract accidentally would be irretrievable, leading to a permanent loss of assets.

Recommendation
Implement a function named withdrawExcessFunds that allows the contract owner to specify the token address and the amount to withdraw. Make sure to add checks to ensure that only non-staked tokens can be withdrawn.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

WSC - 002	No Emergency Unstake Function		
Function Name	-		
Rating	Severity: Medium	Likelihood:	Impact:

Description
The contract does not have an emergency unstake function that would allow users to quickly unstake their tokens in case of an emergency.

Impact
In the event of a critical issue, such as a bug or an exploit, users would not be able to quickly retrieve their staked tokens, potentially leading to a loss of funds.

Recommendation
Add an emergencyUnstake function that allows users to unstake their tokens immediately. This function should only be activatable by the contract owner or through a governance decision.

WSC - 003	No Re-Entrancy Guard		
Function Name	-		
Rating	Severity: Medium	Likelihood:	Impact:

Description
The contract lacks a re-entrancy guard for functions that involve external token transfers, which is a common security measure to prevent re-entrancy attacks.

Impact
The contract is potentially vulnerable to re-entrancy attacks, which could lead to unauthorized withdrawals of tokens.

Recommendation
Use OpenZeppelin's ReentrancyGuard library to add a re-entrancy guard to functions that are vulnerable to such attacks.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

WSC - 004	No Function to Update stakedToken		
Function Name	-		
Rating	Severity: Medium	Likelihood:	Impact:

Description
The contract sets the stakedToken address in the constructor but does not provide a way to update it later.

Impact
If the staked token needs to be changed for any reason, a new contract would have to be deployed, which is inefficient and could lead to loss of user trust.

Recommendation
Add an updateStakedToken function that allows the contract owner to update the stakedToken address. Include appropriate checks and perhaps a time lock for added security.

WSC - 005	No Checks for Zero Address		
Function Name	-		
Rating	Severity: Medium	Likelihood:	Impact:

Description
The constructor sets the owner of the contract but does not include a check to ensure that the owner's address is not the zero address.

Impact:
If the owner's address is set to the zero address, the contract would become unmanageable, and no one would be able to call owner-only functions.

Recommendation
Include a require statement in the constructor to ensure that the owner's address is not the zero address.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

WSC - 006	No Time Lock for Owner-Only Functions		
Function Name	-		
Rating	Severity: Medium	Likelihood:	Impact:

Description
Owner-only functions can be executed immediately without any time lock or community governance.

Impact
Immediate changes to critical parameters could pose a risk to users who have staked tokens.

Recommendation
Implement a time lock mechanism or a multi-signature requirement for critical owner-only functions.

WSC - 007	No Penalty for Early Unstaking		
Function Name	-		
Rating	Severity: Medium	Likelihood:	Impact:

Description
The contract does not impose a penalty for unstaking tokens before the staking period has ended.

Impact
Users could take advantage of this to game the system, which could be detrimental to the intended staking incentives.

Recommendation
Implement a penalty mechanism that deducts a certain percentage of the staked tokens or rewards if a user unstakes early.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Low severity issues

WSC - 008	No Function to Query Unclaimed Stakes		
Function Name	-		
Rating	Severity: Low	Likelihood:	Impact:

Description
The contract does not provide a function to query all unclaimed stakes for a user, which could be useful for frontend development.

Impact
The absence of this function could make it difficult for users to manage their stakes effectively, leading to a poor user experience.

Recommendation
Implement a function named getUnclaimedStakes that returns an array of unclaimed stakes for a given user address.

WSC - 009	No Function to Update Staking Periods		
Function Name	-		
Rating	Severity: Low	Likelihood:	Impact:

Description
The staking periods are hardcoded in the constructor, and there is no function to update them later.

Impact
This limits the contract's flexibility and adaptability to changing market conditions or user needs.

Recommendation
Add a function named updateStakingPeriods that allows the contract owner to update the staking periods. Include appropriate checks and perhaps a time lock for added security.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

WSC - 010	No Function to Retrieve Staking Plans		
Function Name	-		
Rating	Severity: Low	Likelihood:	Impact:

Description
The contract does not provide a function to retrieve the details of available staking plans, which could be useful for frontend development.

Impact
The absence of this function could make frontend development more challenging and result in a less user-friendly interface.

Recommendation
Implement a function named getStakingPlans that returns an array of objects containing the details of each staking plan.

WSC - 011	No Cap on Staking		
Function Name	-		
Rating	Severity: Low	Likelihood:	Impact:

Description
The contract does not set a maximum limit on the amount of tokens that can be staked.

Impact
Unlimited staking might not be desirable depending on the economic model of the token and could lead to centralization of staking power.

Recommendation
Implement a maximum staking limit either per user or for the contract as a whole.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Informational

No issues were found.

Gas Optimization

No issues were found.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Concluding Remarks

While conducting the audits of WERX, it was observed that the system contains medium and low severity issues. The recommendations given will improve the operations of the smart contract.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the Uwerx platform or its product, nor is this audit investment advice.

Notes:

- Please make sure contracts deployed on the mainnet are the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS